



ASCII

Array dan Struct



Array

Array adalah kumpulan variabel yang memiliki tipe data yang sama, disimpan secara berurutan di memori dan diakses menggunakan indeks.

Nilai	10	20	30	40	50
Indeks	0	1	2	3	4

Untuk mendeklarasikan array, tentukan **tipe variabel**, sebutkan **nama array** diikuti tanda kurung siku, dan tentukan **jumlah elemen** yang akan disimpan:

```
TipeData NamaArray[UkuranArray];
```

```
// Contoh deklarasi
int bilangan[5];           // Array integer, 5 elemen
string nama[4];            // Array string, 4 elemen
float nilai[10];           // Array float, 10 elemen

// Deklarasi + Inisialisasi langsung
string buah[4] = {"Jeruk", "Mangga", "Nanas", "ApeL"};

// Deklarasi array tanpa inisialisasi elemen array
int angka[3];
angka[0] = 10;
angka[1] = 19;
angka[2] = 30;
```

Operasi Dasar Array



1. Penyisipan (Insertion)

Memasukkan data ke dalam elemen array, dengan menggunakan operator assignment.

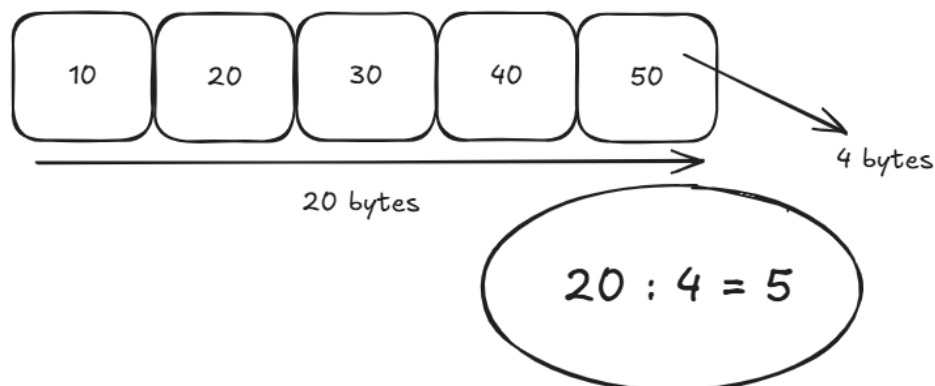
```
buah[4] = "Rambutan";  
for (int i = 0; i < 5; i++) {  
    cout << buah[i] << endl;  
}
```

2. Pengaksesan (Access)

Pengambilan nilai dari elemen dengan indeks tertentu pada array.

```
cout << buah[0] << endl;  
cout << angka[2] << endl;  
cout << endl;  
// For each perulangan  
for (string x : buah) {  
    cout << x << endl;  
}
```

Karena C++ tidak menyimpan informasi panjang array secara otomatis, kita perlu menghitungnya sendiri dengan membagi total ukuran array (dalam bytes) dengan ukuran satu elemennya.





```
int angka[5] = {10, 20, 30, 40, 50};

// sizeof(angka)      = 20 bytes (5 elemen x 4 bytes per int)
// sizeof(angka[0])   = 4 bytes (1 elemen int)

int panjang = sizeof(angka) / sizeof(angka[0]);
cout << "Panjang array: " << panjang << endl;
```

Array 1 Dimensi

Array satu dimensi adalah struktur data linear yang menyimpan sekumpulan elemen bertipe data sama (homogen) dalam satu baris atau urutan berurutan di memori komputer.

CRUD (Create, Read, Update, Delete) pada Array 1 Dimensi

```
int main() {

    int panjang = 0; // Jumlah data mahasiswa saat ini
    string mahasiswa[MAX_MAHASISWA]; // Array 1 dimensi

    int pilihan, index;

    do {
        cout << "\n≡≡≡ MENU PROGRAM DATA MAHASISWA ≡≡≡" << endl;
        cout << "1. Tampilkan Mahasiswa" << endl;
        cout << "2. Tambah Mahasiswa" << endl;
        cout << "3. Ubah Mahasiswa" << endl;
        cout << "4. Hapus Mahasiswa" << endl;
        cout << "5. Keluar" << endl;
        cout << "Pilihan: ";
        cin >> pilihan;

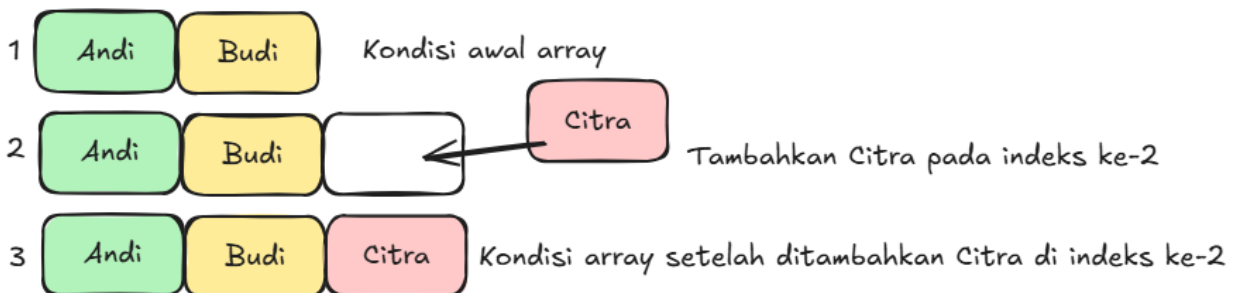
        switch (pilihan) {
```



```
case 1: // READ
    break;
case 2: // CREATE
    break;
case 3: // UPDATE
    break;
case 4: // DELETE
    break;
case 5:
    cout << "Program selesai." << endl;
    break;
default:
    cout << "Pilihan tidak valid." << endl;
}

} while (pilihan != 5);
return 0;
}
```

1. Create

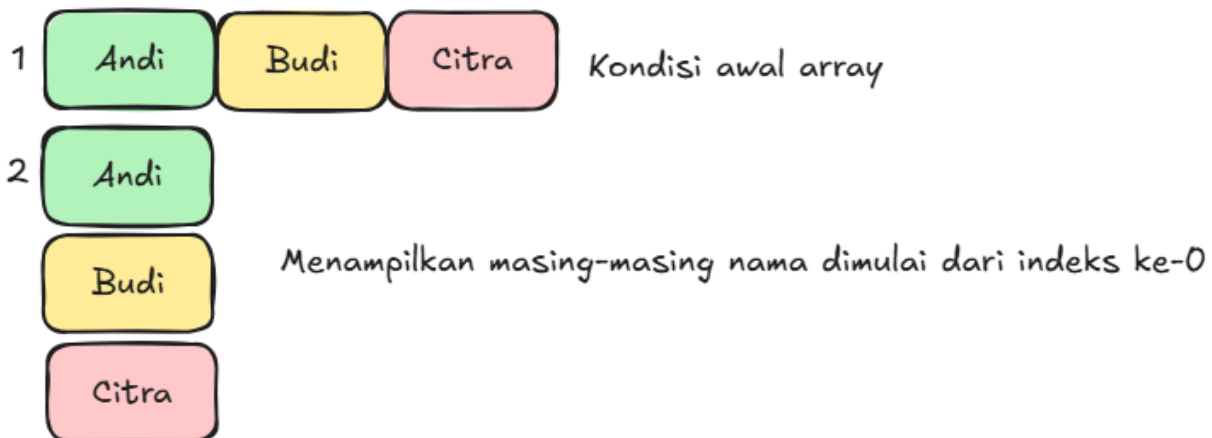


```
if (panjang < MAX_MAHASISWA) {
    cout << "Masukkan nama mahasiswa: ";
    cin.ignore();
    getline(cin, mahasiswa[panjang]);
    panjang++;
}
```



```
    cout << "Mahasiswa berhasil ditambahkan." << endl;
} else {
    cout << "Kapasitas penuh!" << endl;
}
```

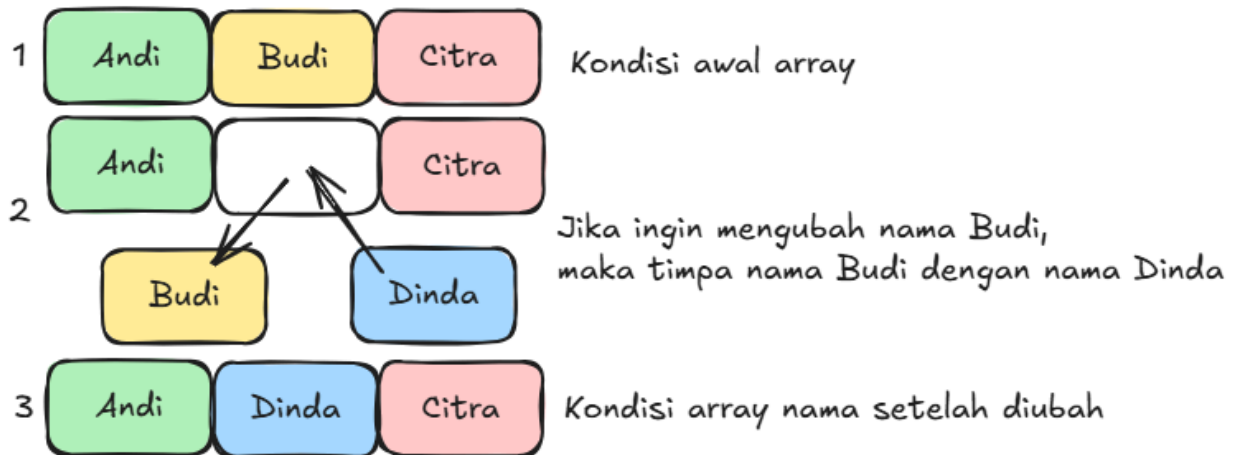
2. Read



```
if (panjang == 0) {
    cout << "Belum ada mahasiswa." << endl;
} else {
    for (int i = 0; i < panjang; i++) {
        cout << "Mahasiswa ke-" << i + 1
            << ": " << mahasiswa[i] << endl;
    }
}
```



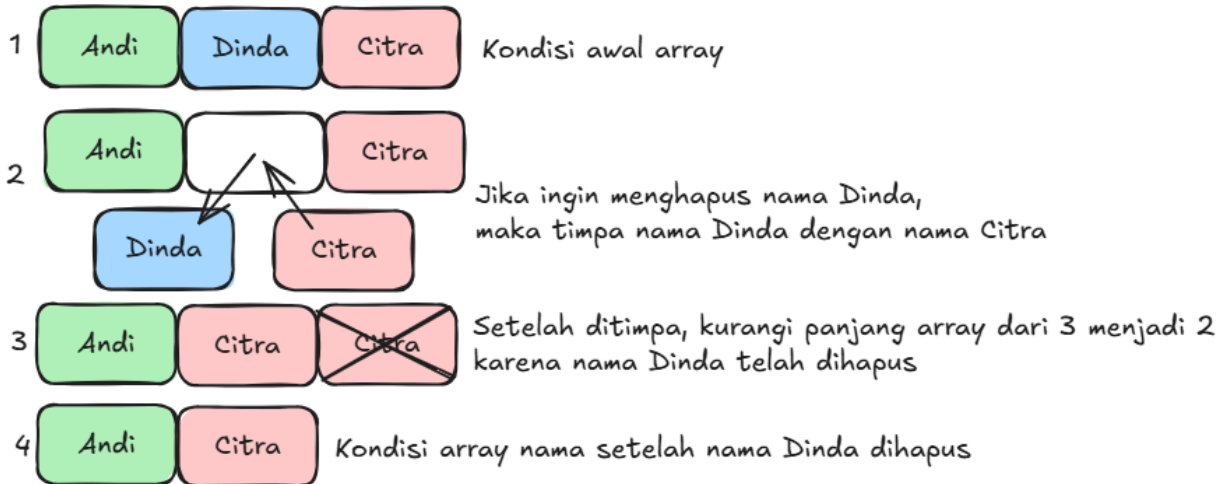
3. Update



```
if (panjang == 0) {  
    cout << "Belum ada mahasiswa untuk diubah." << endl;  
} else {  
    for (int i = 0; i < panjang; i++) {  
        cout << "Mahasiswa ke-" << i + 1  
            << ": " << mahasiswa[i] << endl;  
    }  
  
    cout << "Masukkan nomor mahasiswa yang akan diubah: ";  
    cin >> index;  
  
    if (index > 0 && index <= panjang) {  
        cout << "Masukkan nama baru: ";  
        cin.ignore();  
        getline(cin, mahasiswa[index - 1]);  
        cout << "Data berhasil diubah." << endl;  
    } else {  
        cout << "Nomor tidak valid." << endl;  
    }  
}
```



4. Delete



```
if (panjang == 0) {  
    cout << "Belum ada mahasiswa untuk dihapus." << endl;  
} else {  
    for (int i = 0; i < panjang; i++) {  
        cout << "Mahasiswa ke-" << i + 1  
            << ": " << mahasiswa[i] << endl;  
    }  
  
    cout << "Masukkan nomor mahasiswa yang akan dihapus: ";  
    cin >> index;  
  
    if (index > 0 && index <= panjang) {  
        for (int i = index - 1; i < panjang - 1; i++) {  
            mahasiswa[i] = mahasiswa[i + 1];  
        }  
        panjang--;  
        cout << "Mahasiswa berhasil dihapus." << endl;  
    } else {  
        cout << "Nomor tidak valid." << endl;  
    }  
}
```




Array 2 Dimensi

Array dua dimensi adalah perluasan dari array 1D yang berbentuk tabel (baris × kolom).

TipeData NamaArray[Baris][Kolom];

```
int matriks[3][3] = {
    {1, 2, 3},    // Baris ke-0
    {4, 5, 6},    // Baris ke-1
    {7, 8, 9}     // Baris ke-2
};

// Mengakses elemen
cout << matriks[0][1] << endl;
cout << matriks[2][0] << endl;

// Menampilkan seluruh matriks dengan nested loop
for (int i = 0; i < 3; i++) {    // Loop baris
    for (int j = 0; j < 3; j++) { // Loop kolom
        cout << matriks[i][j] << " ";
    }
    cout << endl;
}

for (auto &baris : matriks) { // Iterasi setiap baris (array 1D)
    for (auto &elemen : baris) { // Iterasi setiap elemen dalam baris
        cout << elemen << " ";
    }
    cout << endl;
}
```



Array Multidimensi

Array multidimensi adalah struktur data kompleks yang terdiri dari array di dalam array, memungkinkan penyimpanan data dalam baris dan kolom (seperti tabel 2D) atau lebih tinggi.

TipeData NamaArray[Tingkat][Baris][Kolom];

```
#include <iostream>
using namespace std;

int main() {

    int nilai[2][2][2] = {
        { {80, 85}, {75, 90} }, // Kelas A
        { {88, 92}, {70, 78} }  // Kelas B
    };

    cout << "≡≡≡ DATA NILAI MAHASISWA ≡≡≡\n\n";

    for (int k = 0; k < 2; k++) {
        cout << "Kelas " << char('A' + k) << endl;

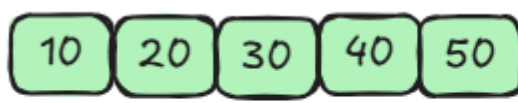
        for (int m = 0; m < 2; m++) {
            cout << "    Mahasiswa " << m + 1 << endl;
            cout << "        UTS : " << nilai[k][m][0] << endl;
            cout << "        UAS : " << nilai[k][m][1] << endl;
        }

        cout << endl;
    }
    return 0;
}
```



Struct

Struct (struktur) dalam C++ adalah tipe data bentukan pengguna (user-defined data type) yang digunakan untuk mengelompokkan beberapa variabel dengan tipe data berbeda ke dalam satu nama. Jika array adalah kumpulan data bertipe **sama**, maka struct adalah kumpulan data yang bisa bertipe **berbeda**. Struct mempermudah pengelolaan data terkait (misalnya data siswa: nama, usia, nilai) agar kode lebih rapi dan efisien, di mana anggota struct diakses menggunakan operator titik (.)



Array



Struct

Deklarasi Struct

Ada dua cara mendeklarasikan struct di C++. Keduanya equivalen, pilih mana yang lebih nyaman.

Cara 1 struct keyword	Cara 2 — typedef struct
<pre>struct nama_struct { tipe_data1 elemen_struct; }; struct Mahasiswa { int nim; string nama; float nilai; };</pre>	<pre>typedef struct { tipe_data1 elemen_struct; }nama_struct; typedef struct { int nim; string nama; float nilai; } Mahasiswa;</pre>



Cara penggunaan atau deklarasi variabel bertipe data struct:

```
Struct Mahasiswa mhs;
```

```
Mahasiswa mhs;
```

Mengakses Elemen Struct

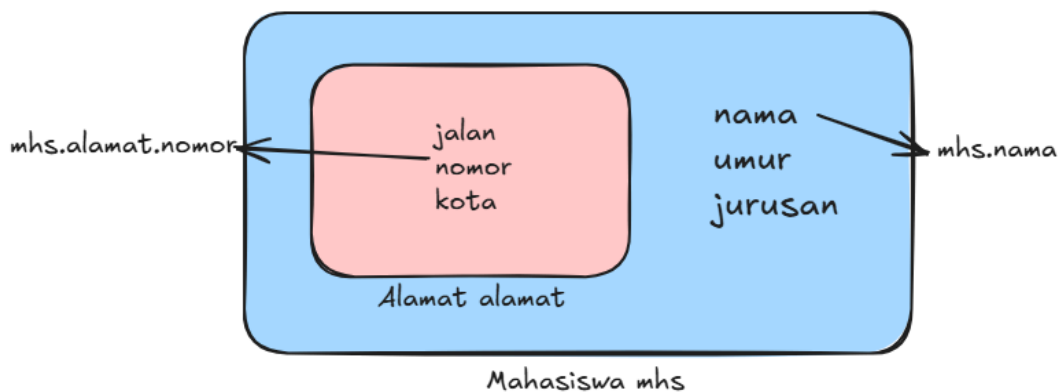
Untuk mengakses atau mengisi field dalam struct, gunakan operator titik (.) setelah nama variabel struct.

```
// Mengisi field (assignment)
mhs.nim    = 2501234;
mhs.nama   = "Bintang";
mhs.nilai  = 88.5;

// Membaca field (akses)
cout << "NIM    : " << mhs.nim    << endl;
cout << "Nama   : " << mhs.nama   << endl;
cout << "Nilai  : " << mhs.nilai  << endl;
```

Nested Struct

Struct bisa berisi struct lain sebagai field-nya. Ini disebut Nested Struct dan sangat berguna untuk data yang hierarkis.





Contoh dalam program:

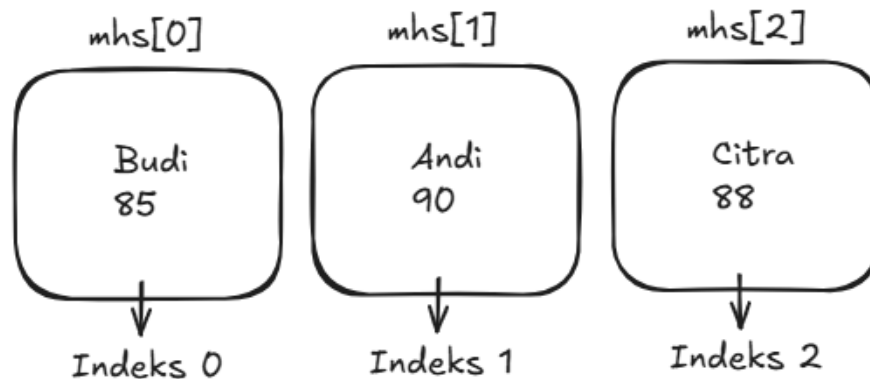
```
struct Alamat {  
    string jalan;  
    int    nomor;  
    string kota;  
};  
  
struct Mahasiswa {  
    string nama;  
    int    umur;  
    string jurusan;  
    Alamat alamat;    // Nested struct  
};  
  
// Penggunaan  
Mahasiswa mhs;  
mhs.nama      = "Intan";  
mhs.umur      = 18;  
mhs.jurusan   = "Informatika";  
mhs.alamat.jalan = "Jl. Kuaro";  
mhs.alamat.nomor = 12;  
mhs.alamat.kota = "Balikpapan";
```

Array of Struct

Array of struct adalah struktur data dalam C++ yang menggabungkan konsep array dan struct, di mana setiap elemen dalam array bertipe struct sehingga dapat menyimpan banyak data yang bersifat kompleks dan saling berkaitan dalam satu kesatuan. Jika struct digunakan untuk merepresentasikan satu objek dengan beberapa atribut berbeda tipe (misalnya NIM, nama, dan nilai pada data mahasiswa), maka array of struct digunakan untuk menyimpan banyak objek tersebut secara berurutan dalam memori. Dengan demikian, array of struct memungkinkan kita mengelola sekumpulan data kompleks secara



lebih terorganisir, efisien, dan mudah diakses menggunakan indeks array serta operator titik (.) untuk mengakses field di dalam struct.



Contoh program

```
struct date {
    int month;
    int day;
    int year;
};

struct mahasiswa{
    string name;
    struct date birthday;
};

struct mahasiswa mhs[10];

mhs[0].name = "Ihsan";
mhs[0].birthday.day = 30;
mhs[0].birthday.month = 1;
mhs[0].birthday.year = 2007;
```

CRUD dengan Array of Struct



1. Menu Program dan Struct

```
#include <iostream>
using namespace std;

struct Alamat {
    string jalan;
    int    nomor;
    string kota;
};

struct Mahasiswa {
    string nama;
    int    umur;
    string jurusan;
    Alamat alamat;
};

#define MAX_MAHASISWA 100
int panjang = 0;
Mahasiswa mhs[MAX_MAHASISWA];

int main() {
    int pilihan, index;

    do {
        cout << "===== " << endl;
        cout << "    Menu Manajemen Mahasiswa    " << endl;
        cout << "===== " << endl;
        cout << "1. Tampilkan Data Mahasiswa" << endl;
        cout << "2. Tambah Data Mahasiswa" << endl;
        cout << "3. Ubah Data Mahasiswa" << endl;
        cout << "4. Hapus Data Mahasiswa" << endl;
        cout << "5. Keluar" << endl;
        cout << "Pilihan: ";
        cin >> pilihan;
    }
```



```
switch (pilihan) {
    case 1: /* ... READ ... */ break;
    case 2: /* ... CREATE ... */ break;
    case 3: /* ... UPDATE ... */ break;
    case 4: /* ... DELETE ... */ break;
    case 5: cout << "Sampai jumpa!" << endl; break;
    default: cout << "Pilihan tidak valid!" << endl; break;
}
} while (pilihan != 5);

return 0;
}
```

2. Read (Tampilkan Data)

```
if (panjang == 0) {
    cout << "Belum ada data mahasiswa." << endl;
} else {
    cout << "Daftar Mahasiswa:" << endl;
    cout << "=====" << endl;
    for (int i = 0; i < panjang; i++) {
        cout << "Mahasiswa ke-" << i + 1 << endl;
        cout << "Nama      : " << mhs[i].nama << endl;
        cout << "Umur      : " << mhs[i].umur << endl;
        cout << "Jurusan   : " << mhs[i].jurusan << endl;
        cout << "Alamat    : " << mhs[i].alamat.jalan
        << " No. " << mhs[i].alamat.nomor
        << ", " << mhs[i].alamat.kota << endl;
        cout << "-----" << endl;
    }
}
```

3. Create (Tambah Data)



```
if (panjang < MAX_MAHASISWA) {
    cout << "Masukkan nama      : ";
    cin.ignore();
    getline(cin, mhs[panjang].nama);

    cout << "Masukkan umur      : ";
    cin >> mhs[panjang].umur;

    cout << "Masukkan jurusan   : ";
    cin.ignore();
    getline(cin, mhs[panjang].jurusan);

    cout << "--- Alamat ---" << endl;
    cout << "Jalan : "; getline(cin, mhs[panjang].alamat.jalan);
    cout << "Nomor : "; cin >> mhs[panjang].alamat.nomor;
    cout << "Kota  : "; cin.ignore(); getline(cin,
mhs[panjang].alamat.kota);

    panjang++;
    cout << "Data berhasil ditambahkan!" << endl;
} else {
    cout << "Kapasitas penuh!" << endl;
}
```

4. Update (Ubah Data)

```
if (panjang == 0) {
    cout << "Belum ada mahasiswa untuk diubah." << endl;
} else {
    cout << "Daftar Mahasiswa" << endl;
    cout << "======" << endl;
    for (int i = 0; i < panjang; i++) {
        cout << "Mahasiswa ke-" << i + 1 << endl;
        cout << "Nama: " << mhs[i].nama << endl;
    }
}
```



```
        cout << endl;
    }
    cout << "Masukkan nomor mahasiswa yang akan diubah: ";
    cin >> index;

    if (index > 0 && index ≤ panjang) {
        cout << "Masukkan nama mahasiswa baru: ";
        cin.ignore();
        getline(cin, mhs[index - 1].nama);
        cout << "Masukkan umur mahasiswa baru: ";
        cin >> mhs[index - 1].umur;
        cout << "Masukkan jurusan mahasiswa baru: ";
        cin.ignore();
        getline(cin, mhs[index - 1].jurusan);
        cout << "Masukkan alamat mahasiswa baru:" << endl;
        cout << "Jalan: "; getline(cin, mhs[index - 1].alamat.jalan);
        cout << "Nomor: "; cin >> mhs[index - 1].alamat.nomor;
        cout << "Kota: "; cin.ignore(); getline(cin, mhs[index -
1].alamat.kota);
        cout << "Mahasiswa berhasil diubah" << endl;
    } else {
        cout << "Nomor mahasiswa tidak valid" << endl;
    }
}
```

5. Delete (Hapus Data)

```
if (panjang == 0) {
    cout << "Belum ada mahasiswa untuk dihapus." << endl;
} else {
    cout << "Daftar Mahasiswa" << endl;
    cout << "======" << endl;
    for (int i = 0; i < panjang; i++) {
        cout << "Mahasiswa ke-" << i + 1 << endl;
    }
}
```



```
        cout << "Nama: " << mhs[i].nama << endl;
        cout << endl;
    }
    cout << "Masukkan nomor mahasiswa yang akan dihapus: ";
    cin >> index;

    if (index > 0 && index ≤ panjang) {
        for (int i = index - 1; i < panjang - 1; i++) {
            mhs[i] = mhs[i + 1];
        }
        panjang--;
        cout << "Mahasiswa berhasil dihapus" << endl;
    } else {
        cout << "Nomor mahasiswa tidak valid" << endl;
    }
}
```



Studi Kasus

Sebuah kantin kampus memiliki 3 menu: Nasi Goreng (Rp15.000), Mie Ayam (Rp12.000), dan Es Teh (Rp5.000). Nasi Goreng dan Mie Ayam sedang tersedia, sedangkan Es Teh sedang habis.

1. Buatlah ilustrasi bagaimana ketiga data menu tersebut disimpan menggunakan array of struct, lengkap dengan tipe data tiap field, isi datanya, dan cara mengakses salah satu field. (Clue: gunakan bool untuk status tersedia/habis).
2. Setelah ilustrasi selesai, implementasikan dalam C++ sesuai ilustrasi yang telah dibuat, lalu tampilkan seluruh menu beserta statusnya. (Bisa dengan cara manual atau loop).